Dynamic-Payload Pick-and-Pour Control for a UR5 Barista Robot: Progress Report

Soumaya El Mansouri, Jingxuan Yang, Tracy Yang
RBE 501, Worcester Polytechnic Institute
Worcester, MA, USA

I. Introduction

A. Problem Description

Pouring is a common manipulation task in food service, laboratory work, and households. Despite this, pouring and handling liquid remains difficult for robotic arms. As liquid leaves the container, the payload held by the arm both loses mass and shifts as the liquid moves. This requires the controller for this task to take into account fluid movement, and dynamic payload, which drastically increases the complexity of a pick and place task. In addition, a poor grasp or inadequate sensing of the poured amount can lead to spills or an overfilled cup.

This project addresses changing payload and the sensing problem for a UR5 manipulator simulated in ROS2 and Gazebo Harmonic.

B. Related Work

Our task combines three problems that the literature has mostly treated separately: modelling the arm itself, perceiving how much liquid has been poured, and deciding how to pour. We organize prior work around these problems and, for each paper, state what we take from it and what we change.

UR5 modelling. Kebria et al. [1] derive the DH parameters, closed-form forward and inverse kinematics, and the Lagrangian dynamics (inertia, Coriolis, and gravity terms) of the UR5, and they verify the model in SimMechanics with a PD controller using feedback linearization. This is the most direct foundation for our project. Using these equations, we compute the Jacobian, manipulability, and joint torques along the pick-and-pour trajectory, and we use their DH table and inverse-kinematics as our starting point. The limitation is that their dynamics assume constant link and payload parameters. In our task, the arm holds a container that loses mass throughout the pour, so a controller built on their model with a fixed payload would see a changing gravity and inertia load as an unmodelled disturbance.

Vision-based liquid volume estimation. Schober et al. [2] estimate the liquid volume in transparent containers from a single RGB image and then execute a pouring motion selected from a simulation on a UR5e. Their pouring strategy of rotating the container about a fixed liquid exit point is useful for the small opening of a coffee container. Their results, however, show why vision alone is not enough for our task: the complete pipeline had RMSE of 21–26 mL for 30–50 mL targets, and the authors note that volume-estimation and simulation errors add up so the setup is not precise enough for laboratory use. Because a pour is selected before it starts and then executed without feedback, an error in the volume estimate cannot be corrected during the pour. We therefore use the camera only to localize the mug and container for grasping and placing, and we take the poured amount from a scale, which gives a direct measurement that can be used to stop the pour.

Force/torque perception of liquids. Luo et al. [3] use a wrist-mounted force/torque sensor on a UR5e to push a whisk through pancake batter and infer its uniformity, liquid level, and water-flour ratio from the resistance torque. They then pour by rotating the wrist at a constant angular speed and use a learned model to set the arm speed from the batter properties. This matters for us for two reasons. First, it shows that a wrist force/torque sensor on this class of arm gives measurements precise enough to be useful (liquid level within about 3–4% error). Second, it uses the sensor at the same location we plan to, which supports using it to track the weight of the container in the gripper as it empties. Their pour, however, is open loop with respect to the amount poured: the properties are measured before pouring, and nothing is measured during the pour to decide when to stop. We close this loop by feeding the sensor and scale measurements back into both the stopping decision and the controller.

Learned pouring policies. Babaians et al. [4] train PourNet, a reinforcement-learning policy with curiosity-based exploration and curriculum learning in simulation, and execute its motion on a Kinova Movo with a nonlinear model predictive controller. It is the work closest to ours in goal. Its second arm holds the target container and measures its weight with a force/torque sensor, and the pour is based on this feedback, with deviations of a few grams across liquids. It also gives a practical recipe for the measurement itself using an IMU for gravity compensation and a low-pass filter on the force/torque signal (1 Hz cut-off), which we can adapt for our scale and wrist sensor. The cost is that the policy needs millions of simulated training steps and treats the pour as end-effector planning, so it does not model how the arm's own dynamics change as the held container loses mass. For a course project

TABLE I
COMPARISON OF RELATED WORK AND THIS PROJECT

Work	Sensing	Pour control	Time-varying load modelled
[1]	–	PD tracking	No
[2]	RGB camera	Simulation-selected, no feedback	No
[3]	Wrist F/T	Open-loop	No
[4]	F/T (weight of target)	RL + NMPC	Learned implicitly
Ours	Camera, scale, wrist F/T	Model-based, dynamic load	Yes

with limited simulation time, a model-based controller whose structure is known and which can be analyzed with the Jacobian and manipulability tools from class is a better fit than a trained policy.

Comparison and our approach. Table I compares these works with ours. Taken together, the prior work establishes (i) a complete UR5 model [1], (ii) that vision alone is too imprecise for the poured amount [2], (iii) that wrist force/torque sensing carries useful information about the liquid [3], and (iv) that weight feedback from the target container gives accurate pours [4]. What none of them does is model the arm carrying a payload whose mass changes during the pour and design the controller around it. Our work fills this gap: we extend the UR5 model of [1] with a time-varying payload, combine a camera (object localization), a scale (poured mass), and a wrist force/torque sensor (payload mass) for feedback, and use a model-based controller that compensates for the changing load. We evaluate this on the complete pick-up-and-pour sequence in ROS2 and Gazebo and examine manipulability and singularities along the trajectory.

II. METHODOLOGY

A. Robot Sequence

The test sequence is a complete beverage-preparation task in which the arm carries three different payloads: an empty mug, a pitcher whose mass decreases during the pour, and a filled mug. The workspace consists of a table holding the mug, the pitcher of coffee, and a scale. Starting from a fixed home configuration clear of the table, the arm performs the following steps:

[**INSERT GAZEBO IMAGES**]

1) *Mug pick-up:* the camera localizes the mug and the arm grasps it.
2) *Mug placement:* the arm places the mug on the scale and releases it.
3) *Pitcher pick-up:* the camera localizes the pitcher, the arm grasps it, and the wrist force/torque sensor measures its initial mass.
4) *Pour:* the arm positions the pitcher above the mug and tilts it. Throughout the pour, the payload mass estimated from the wrist sensor is passed to ends when the scale reports the target mass, at which point the arm returns the pitcher upright.
5) *Pitcher return:* the arm sets the empty pitcher down on the table and releases
6) *Serving:* the arm grasps the filled mug and places it at a serving location.

Each grasp follows the same pre-grasp pose near the object, approaches it, grasps it, and lifts it clear of the surface.

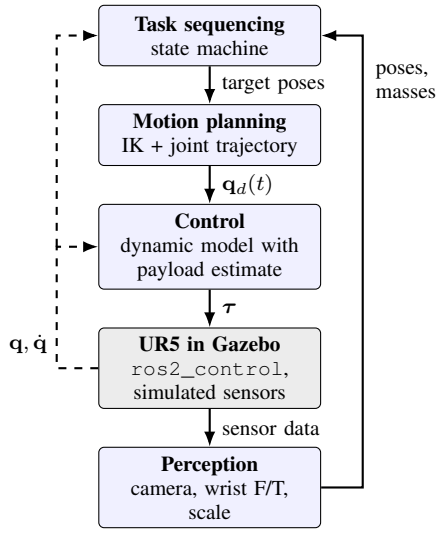


Fig. 1. Control pipeline. Solid arrows show the forward flow of commands and data between stages; dashed arrows show joint-state feedback from the robot.

B. Pipeline

The system is implemented in ROS 2 Jazzy with Gazebo Harmonic. The simulation contains the UR5 arm, the table with the mug, pitcher, and scale, a camera, and a wrist force/torque sensor; the arm is driven through `ros2_control`, and the remaining simulator topics are exchanged with ROS 2 through `ros_gz_bridge`. The pipeline consists of four stages:

1) *Task sequencing:* a state machine steps through the sequence above, deciding when to grasp, release, and stop the pour based on the sensing outputs.
2) *Motion planning:* for each step, inverse kinematics converts the target end-effector poses into joint configurations, which are interpolated into a joint trajectory.
3) *Control:* the controller tracks this trajectory. The arm's dynamic model is updated with the payload mass from the force/torque sensor, so the controller compensates for the pitcher's weight as it changes during the pour.
4) *Sensing:* three sensors observe the task. The camera estimates the poses of the mug and pitcher, the wrist force/torque sensor estimates the mass of the grasped object, and the scale measures the mass poured into the mug. The joint encoders report the arm's joint positions and velocities.

TABLE II
UR5 DH PARAMETERS (`UR5_PARAMS.PY`)

i	a_i [m]	α_i	d_i [m]	θ_i
1	0	$\pi/2$	0.08916	θ_1
2	-0.425	0	0	θ_2
3	-0.39225	0	0	θ_3
4	0	$\pi/2$	0.10915	θ_4
5	0	$-\pi/2$	0.09465	θ_5
6	0	0	0.0823	θ_6

Joint states from the measurements are fed back to the controller and the state machine at each control step as seen in Figure 1.

C. Kinematic Model

1) Parameters and Conventions: Forward kinematics, the Jacobian, inverse kinematics, and manipulability are all implemented in `kinematics.py`, which reads its parameters from `ur5_params.py`. We use the manufacturer's DH table (Table II), whose negative a_2 and a_3 match the joint angles reported by the UR controller; Kebria et al. [1] list them as positive, which describes a mirrored arm. Frames $\{0\}$ and $\{6\}$ are the ROS `base` and `tool0` frames. The home position, with the arm pointing straight up, is $\mathbf{q}_{\text{home}} = [0, -\frac{\pi}{2}, 0, -\frac{\pi}{2}, 0, 0]^T$.

2) Jacobian: The geometric Jacobian maps joint velocities to the end-effector linear and angular velocities:

$$\begin{bmatrix}\mathbf{v}\\ \boldsymbol{\omega}\end{bmatrix} = J(\mathbf{q})\dot{\mathbf{q}}. \tag{1}$$

For each revolute joint, the Jacobian column is

$$J_i = \begin{bmatrix}\mathbf{z}_{i-1} \times (\mathbf{p}_e - \mathbf{p}_{i-1})\\ \mathbf{z}_{i-1}\end{bmatrix}. \tag{2}$$

The Jacobian is implemented in `kinematics.py` and used for Cartesian velocity control.

3) Forward Kinematics: Forward kinematics computes the end-effector pose from the joint configuration using the product-of-exponentials formulation:

$$T(\mathbf{q}) = e^{[\mathcal{S}_1]\theta_1} \cdots e^{[\mathcal{S}_6]\theta_6} M. \tag{3}$$

Here, $\mathcal{S}_i$ represents the screw axis of joint i, and M is the home configuration of the end-effector. The resulting transformation provides the tool position and orientation relative to the robot base.

4) Inverse Kinematics: `solve_ik()` implements the closed-form solution of [1]. For a desired pose ${}^0T_6$ with tool axes $\hat{\mathbf{x}}, \hat{\mathbf{y}}, \hat{\mathbf{z}}$ and position $\mathbf{p}$, the wrist center $\mathbf{p}_5 = \mathbf{p} - d_6\hat{\mathbf{z}}$ gives

$$\theta_1 = \operatorname{atan2}(p_{5y}, p_{5x}) \pm \cos^{-1}\frac{d_4}{\sqrt{p_{5x}^2 + p_{5y}^2}} + \frac{\pi}{2}, \tag{4}$$

$$\theta_5 = \pm\cos^{-1}\frac{p_x s_1 - p_y c_1 - d_4}{d_6}, \tag{5}$$

$$\theta_6 = \operatorname{atan2}\Big(\frac{-\hat{y}_x s_1 + \hat{y}_y c_1}{s_5}, \frac{\hat{x}_x s_1 - \hat{x}_y c_1}{s_5}\Big). \tag{6}$$

When $s_5 = 0$, θ_6 is undetermined and the current value is kept. Joints 2–4 form a planar 3R arm. With (x_{14}, y_{14}) the in-plane position of ${}^1T_4 = ({}^0T_1)^{-1}\,{}^0T_6\,({}^4T_6)^{-1}$,

$$\theta_3 = \pm\cos^{-1}\frac{x_{14}^2 + y_{14}^2 - a_2^2 - a_3^2}{2a_2a_3}, \tag{7}$$

$$\theta_2 = \operatorname{atan2}(y_{14}, x_{14}) - \operatorname{atan2}(a_3 s_3,\ a_2 + a_3 c_3), \tag{8}$$

$$\theta_4 = \operatorname{atan2}\big({}^1R_{4,21}, {}^1R_{4,11}\big) - \theta_2 - \theta_3. \tag{9}$$

The three $\pm$ choices give up to eight solutions. `solve_ik_closest()` returns the one nearest the current joint state, which keeps the arm in the same configuration branch between waypoints.

5) Manipulability: Because the UR5 Jacobian is square, Yoshikawa's measure reduces to $|\det J|$, which factors in closed form (`manipulability()`):

$$w(\mathbf{q}) = \big|\, a_2 a_3\, s_3\, s_5\, (a_2 c_2 + a_3 c_{23} + d_5 s_{234})\big| \quad [\mathrm{m}^3]. \tag{10}$$

The three factors vanish at the elbow ($s_3 = 0$, arm fully extended or folded), wrist ($s_5 = 0$, axes 4 and 6 parallel), and shoulder (wrist center on the plane of axes 1 and 2) singularities, and `singularities()` reports which ones are near. The home position has $\theta_3 = \theta_5 = 0$ and is therefore singular ($w = 0$), so the arm leaves it with joint-space motion. During the pour, the pitcher tilt must keep θ_5 away from 0 and π.

6) Implementation and Verification: `ur5_kinematics_node.py` publishes $w(\mathbf{q})$ from `/joint_states`, warns when $w < 10^{-3}\,\mathrm{m}^3$, and returns the closest IK solution for target poses in `base`. The unit tests in `test_kinematics.py` confirm, over 300 random configurations, that every IK branch reproduces the target pose to within 10^{-8} and that Eq. (10) matches $\sqrt{\det(JJ^T)}$ to within 10^{-10}.

D. Dynamic Model with Time-Varying Payload

The arm obeys

$$M(\mathbf{q}, \boldsymbol{\pi}_p)\ddot{\mathbf{q}} + C(\mathbf{q}, \dot{\mathbf{q}}, \boldsymbol{\pi}_p)\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q}, \boldsymbol{\pi}_p) = \boldsymbol{\tau}, \tag{11}$$

where $\boldsymbol{\pi}_p = (m_p,\ \mathbf{h}_p = m_p\mathbf{c}_p,\ I_p)$ are the mass, first moment, and inertia (about the `tool0` origin) of thegrasped object. `dynamics.py` evaluates Eq. (11) with recursive Newton–Euler. The link parameters are read from the same URDF that Gazebo simulates, so the model and the simulation differ only in $\boldsymbol{\pi}_p$.

1) Payload Contribution: Newton–Euler is linear in each body's inertial parameters, so the payload enters additively through the Jacobian J of the kinematic model:

$$\boldsymbol{\tau} = \boldsymbol{\tau}_{\text{arm}}(\mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}}) + J(\mathbf{q})^T\mathcal{W}_p, \tag{12}$$

$$\mathcal{W}_p = \begin{bmatrix} m_p(\mathbf{a} - \mathbf{g}_0) + \dot{\boldsymbol{\omega}} \times \mathbf{h}_p + \boldsymbol{\omega} \times (\boldsymbol{\omega} \times \mathbf{h}_p)\\ I_p\dot{\boldsymbol{\omega}} + \boldsymbol{\omega} \times I_p\boldsymbol{\omega} + \mathbf{h}_p \times (\mathbf{a} - \mathbf{g}_0)\end{bmatrix},$$

where $\mathbf{a}$ and $\boldsymbol{\omega}$ are the linear acceleration and angular velocity of `tool0` and $\mathbf{g}_0 = [0, 0, -9.81]^T$. Updating $\boldsymbol{\pi}_p$ therefore only changes the last link, and `set_payload()` runs at every control step.

2) Draining Pitcher: We model the pitcher as a rigid shell $(m_s, \mathbf{c}_s, I_s)$ plus a cylinder of liquid of radius r and density ρ standing on the inner bottom $\mathbf{b}$ along the pitcher axis $\hat{\mathbf{a}}$ (`payload.Pitcher`). For a remaining liquid mass $m_l(t)$,

$$h_l = \frac{m_l}{\rho \pi r^2}, \qquad \mathbf{c}_l = \mathbf{b} + \tfrac{h_l}{2}\hat{\mathbf{a}}, \tag{13}$$
$$I_l = I_\perp E_3 + (I_\parallel - I_\perp)\,\hat{\mathbf{a}}\hat{\mathbf{a}}^T,$$
$$I_\parallel = \tfrac{1}{2} m_l r^2, \qquad I_\perp = \tfrac{m_l}{12}(3r^2 + h_l^2),$$

and the shell and liquid combine into $\boldsymbol{\pi}_p(t)$ by summing first moments and parallel-axis inertias. As the pitcher empties, both m_p and the center of mass drop. The pour is slow and the liquid leaves the spout with roughly the pitcher's velocity, so the outflow adds no thrust term and Eq. (11) holds with $\boldsymbol{\pi}_p(t)$. We neglect slosh and the shift of the liquid's center of mass with tilt.

3) Payload Estimation: The wrist sensor measures the force $\mathbf{f}$ that the wrist applies to the payload, $\mathbf{f} = m_p R^T(\mathbf{a} - \mathbf{g}_0)$, with R the sensor orientation from forward kinematics. Projecting onto $\mathbf{e} = R^T(\mathbf{a} - \mathbf{g}_0)$ gives

$$\hat{m}_p = \frac{\mathbf{f}^T \mathbf{e}}{\|\mathbf{e}\|^2}, \qquad \hat{m}_l = \hat{m}_p - m_s, \tag{14}$$

which `MassEstimator` low-pass filters at 1 Hz [4] after subtracting the reading with the gripper empty. The scale gives an independent value $m_l(t) = m_l(0) - m_{\text{scale}}(t)$, used to stop the pour and to check $\hat{m}_l$. With the upper arm vertical and the forearm horizontal, draining 0.8 kg of liquid lowers the shoulder and elbow gravity torques from 22.6 to 18.8 N m (17%). A fixed-payload model would carry this error into the controller.

4) Verification: The unit tests in `test_dynamics.py` check that

- M matches Eq. (8) of [1] and is positive definite;
- $\dot{\mathbf{g}}$ equals the gradient of the potential energy;
- $\dot{M} - 2C$ is skew-symmetric;
- the payload torque equals $J^T \mathcal{W}_p$ in Eq. (12), with J and $\mathcal{W}_p$ computed independently;
- $\hat{m}_p$ converges to the true mass under noise and motion.

E. Controller

A Jacobian-based Cartesian position controller is implemented to control the UR5 end-effector. The desired Cartesian velocity is calculated using proportional feedback:

$$\mathbf{v}_d = K_p(\mathbf{p}_d - \mathbf{p}). \tag{15}$$

The corresponding joint velocities are computed using damped least squares:

$$\dot{\mathbf{q}} = J_v^T (J_v J_v^T + \lambda^2 I)^{-1} \mathbf{v}_d, \tag{16}$$

where J_v is the translational Jacobian and λ is the damping factor. The resulting joint velocity commands are sent to Gazebo through `ros2_control`.

REFERENCES

[1] P. M. Kebria, S. Al-Wais, H. Abdi, and S. Nahavandi, "Kinematic and dynamic modelling of UR5 manipulator," in *Proc. IEEE Int. Conf. Systems, Man, and Cybernetics (SMC)*, Budapest, Hungary, 2016, pp. 4229–4234.

[2] D. Schober, R. Güldenring, J. Love, and L. Nalpantidis, "Vision-based robot manipulation of transparent liquid containers in a laboratory setting," arXiv:2404.16529, 2024.

[3] X. Luo, S. Jin, H.-J. Huang, and W. Yuan, "An intelligent robotic system for perceptive pancake batter stirring and precise pouring," arXiv:2407.01755, 2024.

[4] E. Babaians, T. Sharma, M. Karimi, S. Sharifzadeh, and E. Steinbach, "PourNet: Robust robotic pouring through curriculum and curiosity-based reinforcement learning," in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, Kyoto, Japan, 2022, pp. 9332–9339.